# IT Support Specialist

## Core Competencies Tutorial

Simple A1-level English explanations with examples and sample interview answers.

Read this guide before your interview. Practice saying the sample answers aloud.

This guide explains every skill listed in your Core Competencies section. For each topic, you will find: what it is, why it is important, how it works, key tools, a possible interview question, and a sample answer you can use. All explanations are in simple English so you can understand and remember them easily.

### CONTENTS

TOPIC 1

## Merchant Onboarding & API Integration

### What is this?

**Merchant onboarding** means helping a new business (merchant) connect to a payment system. When a store or website wants to accept payments, they need to integrate with the payment company. This process is called "onboarding." As a support specialist, your job is to guide the merchant step by step: give them API keys, help them set up the test environment, explain how the API works, check their integration, and finally move them to the live (production) environment.

**API (Application Programming Interface)** is a way for two systems to talk to each other. For example, when a customer clicks "Pay" on a website, the website sends a request through the API to the payment company. The payment company processes the payment and sends back a response: "success" or "failed." Your job is to make sure this connection works correctly.

### Why is it important?

Without proper onboarding, merchants cannot accept payments. If the integration is broken, the merchant loses money and customers get angry. The payment company also loses business. A good onboarding experience makes merchants happy and they stay with the company for a long time. This is one of the most important responsibilities for an IT Support Specialist at a payment company.

### How does it work?

The onboarding process has several steps:

1. **Registration:** The merchant signs up and creates an account.
2. **Credential Setup:** You give the merchant API keys and access credentials.
3. **Test Environment:** The merchant connects to the test (sandbox) environment. They can test payments without real money.
4. **Integration Support:** The merchant's developers write code to connect to the API. If they have problems, you help them. You answer questions, check their requests, and explain error codes.
5. **Testing:** You create test cases. You test different scenarios: successful payment, failed payment, refund, cancellation.
6. **Go-Live:** After all tests pass, you move the merchant to the production (live) environment. Now they can accept real payments from real customers.

**Key Tools / Keywords:** API keys, Sandbox/Test environment, Postman, Test cases, Error codes, JSON, HTTP requests

**Possible Interview Question:**

Q: Can you explain the merchant onboarding process?

**Sample Answer:**

Of course. The merchant onboarding process has 6 main steps. First, the merchant registers and creates an account. Then I set up their API credentials and give them access to the test environment. The merchant's developers write code to connect to our API. During this time, I provide hands-on support: I answer their questions, help them understand error codes, and check their API requests. After the integration is ready, I create test cases and we test different scenarios together: successful payments, failed payments, refunds. When all tests pass, I move the merchant to the production environment so they can accept real payments.

Tip: Always say "test environment" and "production environment" - this shows you understand the process.

TOPIC 2

## Incident Management & Troubleshooting

### What is this?

**Incident management** means handling problems when something goes wrong. For example: a merchant cannot process payments, a transaction fails, the system is slow, or a merchant gets an error message. When a problem happens, it is called an "incident." Your job is to receive the incident report, investigate the problem, find the root cause, fix it, and inform the merchant.

**Troubleshooting** is the process of finding and fixing problems step by step. You start with the most common causes and check each one. For example, if a payment fails: Is the internet connection working? Are the API credentials correct? Is the merchant's code sending the right data? You check each possibility until you find the problem.

### Why is it important?

When merchants have problems, they cannot do business. Every minute of downtime means lost money. Fast incident management is critical for a payment company. If you fix problems quickly, merchants trust the company. If you are slow, merchants leave. Good troubleshooting skills also mean you solve problems permanently, not just temporarily. This prevents the same problem from happening again.

### How does it work?

The incident management process:

1. **Receive:** A merchant reports a problem (via email, ticket, or phone). You create an incident ticket with details: who, what, when, error message.
2. **Categorize:** How serious is the problem? Critical (system is down), High (some merchants affected), Medium (one merchant has a problem), Low (question or small issue).
3. **Investigate:** You check logs, test the system, reproduce the error. You ask the merchant for more information if needed.
4. **Fix:** You apply a solution. This might be: fixing a configuration, resetting credentials, escalating to the development team, or providing a workaround.
5. **Verify:** You confirm with the merchant that the problem is fixed.
6. **Document:** You write what happened, what you did, and how it was fixed. This helps if the same problem happens again.

**Key Tools / Keywords:** Ticket system (Jira), Priority levels (Critical/High/Medium/Low), Escalation, SLA

**Possible Interview Question:**

Q: How do you handle an incident when a merchant reports a payment failure?

Sample Answer:

First, I create a ticket with all the details: the merchant name, the error message, and when the problem started. Then I categorize the priority. If it is critical, I start investigating immediately. I check the system logs to see what happened. I try to reproduce the error in the test environment. If I find the root cause, I fix it - for example, if it is a credential problem, I reset the credentials. If it is a system bug, I escalate to the development team and provide them with all the details they need. After the fix, I contact the merchant to confirm the problem is resolved. Finally, I document the incident in our knowledge base for future reference.

Tip: Use the word "escalate" - it shows you know when to involve senior team members.

TOPIC 3

## Log Analysis & Root Cause Investigation

### What is this?

**Log analysis** means reading system logs to understand what happened. A log is a record of events that the system writes automatically. For example, every API request, every error, every payment transaction is recorded in a log. When something goes wrong, you read the logs to find out why.

**Root cause investigation** means finding the real reason why a problem happened. For example, a payment failed. The error message says "timeout." But "timeout" is just a symptom. The root cause might be: the database was slow, the network was down, or the merchant's server was overloaded. Your job is to find the ROOT cause, not just the symptom.

### Why is it important?

Without log analysis, you are guessing. You might fix the wrong thing and the problem comes back. Log analysis gives you facts and evidence. Root cause investigation is important because if you only fix the symptom, the same problem will happen again. For example, if you restart a server when it crashes but do not find out WHY it crashed, it will crash again tomorrow. Finding the root cause prevents future problems.

### How does it work?

How to analyze logs:

1. **Identify the time:** When exactly did the problem happen? Look at the timestamp.
2. **Search for errors:** Use keywords like "ERROR", "FAIL", "EXCEPTION", "TIMEOUT" to find relevant log entries.
3. **Follow the chain:** Start from the error and trace backwards. What happened before the error? What was the system doing?
4. **Check related systems:** If the API returned an error, check: was the database working? Was the network OK? Was the request format correct?
5. **Find the root cause:** Ask "why" multiple times. The payment failed. Why? Timeout. Why? Database was slow. Why? A query was running too long. Why? Missing index. That is the root cause!
6. **Document:** Write down what you found and share it with the team.

**Key Tools / Keywords:** ELK Stack (Elasticsearch, Logstash, Kibana), grep, log files, Kibana dashboard

**Possible Interview Question:**

Q: How do you use log analysis to find the root cause of a problem?

**Sample Answer:**

When I receive an incident report, the first thing I do is check the logs. I open our ELK Stack, which includes Kibana for log visualization. I search for the time of the incident and filter by the merchant ID. I look for error messages like "ERROR" or "EXCEPTION." When I find the error, I trace backwards to understand what happened before it. For example, if I see a "connection timeout" error, I check if the database was responding at that time, if the network was stable, and if the request was correct. I keep asking "why" until I find the real root cause. Once I identify it, I fix the problem and document my findings so the team can learn from it.

Tip: Always mention ELK Stack if you have used it - it is a very common tool in fintech.

TOPIC 4

## Network Tools (ping, telnet, ipconfig, traceroute)

### What is this?

These are simple command-line tools that help you check if a network connection is working. Think of them as basic tools for checking if two computers can talk to each other.

**ping:** Sends a small message to another computer and waits for a reply. It tells you: Can I reach this computer? How long does it take? Example: ping google.com - you send 4 messages and get 4 replies. If you do not get a reply, the computer is not reachable.

**telnet:** Connects to a specific port on another computer. It tells you: Is this service (port) open and working? Example: telnet api.payment.com 443 - checks if the API is accepting connections on port 443.

**ipconfig (Windows) / ifconfig (Linux):** Shows your computer's network configuration. It tells you: What is my IP address? What is my DNS server? This helps you check if your network settings are correct.

**traceroute (Linux) / tracert (Windows):** Shows the path that data takes from your computer to the destination. It tells you: Which routers does the data pass through? Where is the connection slow or broken? Example: traceroute api.payment.com - shows every "hop" (router) between you and the API.

### Why is it important?

Many problems are caused by network issues. A merchant might say "your API is not working" but the real problem is their network cannot reach your server. With these tools, you can quickly check: Is it a network problem or an application problem? This saves time because you do not waste time looking at application code when the problem is actually in the network.

### How does it work?

When a merchant reports a connection problem, you use these tools step by step:

1. **ping the server:** Can you reach the API server? If yes, the server is alive. If no, the server might be down or

blocked by a firewall.
2. **telnet the port:** Is the API service running on the correct port? For example, HTTPS uses port 443. If telnet fails, the service is not running.
3. **traceroute:** Where is the connection failing? If hop 3 shows a timeout, the problem is at hop 3 (a router or firewall).
4. **ipconfig:** Check the merchant's DNS settings. Maybe they are using a DNS server that does not resolve your domain correctly.
5. **Report findings:** Tell the merchant: "The problem is on your network at hop 3. Please contact your network administrator."

**Key Tools / Keywords:** ping, telnet, traceroute/tracert, ipconfig/ifconfig, nslookup, DNS, Firewall

**Possible Interview Question:**

Q: A merchant says they cannot connect to our API. How do you troubleshoot?

**Sample Answer:**

First, I ask the merchant for the error message and the time it happened. Then I start with basic network checks. I ping our API server from our side to confirm it is up and running. Then I ask the merchant to ping our server from their side. If they cannot ping it, I ask them to run a traceroute to see where the connection fails. If they can ping it but cannot connect to the API, I ask them to telnet our API port, usually port 443 for HTTPS. If telnet fails, it might be a firewall issue on their side. I also ask them to check their DNS settings with ipconfig and try using a public DNS. Based on the results, I either fix the issue on our side or guide the merchant to fix it on their network side.

Tip: Always say "I check from our side first, then ask the merchant to check from their side." This is systematic.

TOPIC 5

## REST API Testing & Postman

### What is this?

**REST API** is the most common way for systems to communicate over the internet. REST uses standard HTTP methods: GET (read data), POST (send/create data), PUT (update data), DELETE (remove data). When a merchant integrates with a payment system, they use REST API to send payment requests.

**Postman** is a tool for testing APIs without writing code. You enter the API URL, select the method (GET/POST), add headers and body data, and click "Send." Postman shows you the response: status code, response body, time taken. It is like a "browser for APIs" - you can test any API endpoint quickly and easily.

### Why is it important?

When a merchant has a problem with the API, you need to test it yourself. Postman lets you reproduce the exact same request the merchant is making. If the API works in Postman, the problem is on the merchant's side (their code). If the API also fails in Postman, the problem is on the server side. This helps you determine where the problem is and fix it faster.

### How does it work?

How to test an API with Postman:

1. **Open Postman** and create a new request.
2. **Select the method:** GET, POST, PUT, or DELETE.
3. **Enter the URL:** For example, https://api.cibpay.az/v1/payments
4. **Add headers:** For example, Authorization: Bearer YOUR_API_KEY, Content-Type: application/json
5. **Add body (for POST):** The data you want to send. For example: {"amount": 100, "currency": "AZN", "merchant_id": "123"}
6. **Click Send** and check the response.
7. **Analyze the response:** Status code 200 = success, 400 = bad request, 401 = unauthorized, 500 = server error.

**Key Tools / Keywords:** Postman, HTTP methods (GET/POST/PUT/DELETE), Status codes (200/400/401/500), JSON, Headers, API keys

**Possible Interview Question:**

Q: How do you use Postman to troubleshoot API issues?

**Sample Answer:**

When a merchant reports an API issue, I first reproduce the request in Postman. I enter the same URL, method, headers, and body that the merchant is using. I check the response: if I get a 200 success, then the problem is on the merchant's side - maybe their code has a bug or they are using wrong headers. If I also get an error, I check the status code and response body. For example, if I get a 401, the API key might be wrong. If I get a 500, it might be a server-side issue and I escalate to the development team. I also save my test requests in Postman collections for future use, so I can quickly retest when similar issues are reported.

Tip: Mention status codes: 200 = OK, 400 = Bad Request, 401 = Unauthorized, 404 = Not Found, 500 = Server Error.

TOPIC 6

## Back-Office Administration

### What is this?

**Back-office administration** means doing internal tasks in the company's admin panel that are not visible to customers but are necessary for the business to run. For a payment company, this includes: creating merchant accounts, setting up API credentials, managing permissions, configuring payment settings, and monitoring transactions.

Think of it like the "control room" of the payment company. While merchants see the public API, the back-office is where the company's staff manages everything behind the scenes.

### Why is it important?

Without back-office administration, merchants cannot be set up, transactions cannot be monitored, and problems cannot be investigated. The back-office is essential for daily operations. A support specialist uses the back-office every day to help merchants: reset their credentials, change their settings, check their transaction history, and configure new services.

### How does it work?

Common back-office tasks:

1. **Merchant Account Management:** Create new accounts, update merchant details, activate or deactivate accounts.
2. **Credential Management:** Generate API keys, reset passwords, manage access permissions. This is very important for security.
3. **Transaction Monitoring:** View transaction history, check payment statuses, investigate disputed transactions.
4. **Configuration:** Set up payment methods, currency settings, webhook URLs (where to send payment notifications).
5. **Reporting:** Generate reports on transaction volumes, success rates, error rates.

**Key Tools / Keywords:** Admin panel, Dashboard, Webhooks, API keys, Transaction reports, Merchant settings

#### Possible Interview Question:

Q: What kind of back-office tasks have you done?

#### Sample Answer:

In my previous roles, I regularly used the admin panel for various back-office tasks. I created and managed merchant accounts, set up API credentials, and configured payment settings. I managed access permissions - for example, giving a merchant read-only access or full access depending on their needs. I also monitored transactions through the admin dashboard, checking payment statuses and investigating failed or disputed transactions. When a merchant needed to change their webhook URL or add a new payment method, I configured these settings in the back-office. Security was always a priority - I followed strict protocols when handling credentials and access permissions.

Tip: Always mention "security" and "strict protocols" when talking about credential management.

TOPIC 7

## Payment Processing Workflows

### What is this?

**Payment processing** is the complete journey of a payment from start to finish. When a customer clicks "Pay" on a website, many things happen in seconds:

1. The merchant's website sends a payment request to the payment company (CIBPay).
2. CIBPay checks: Is this merchant active? Are the credentials valid?
3. CIBPay sends the payment to the bank or card network (Visa, Mastercard).
4. The bank checks: Does the customer have enough money? Is the card valid?
5. The bank sends back: "Approved" or "Declined."
6. CIBPay sends the result to the merchant.
7. The merchant shows "Payment Successful" or "Payment Failed" to the customer.

### Why is it important?

As an IT Support Specialist at a payment company, you must understand this workflow. When a merchant reports a problem, you need to know WHERE in the workflow the problem is. Is the request not reaching CIBPay? Did CIBPay reject it? Did the bank decline it? Understanding the workflow helps you find the problem quickly and tell the merchant exactly what happened.

### How does it work?

There are also additional workflows you should know:

**Refund:** A merchant wants to return money to a customer. The merchant sends a refund request through the API. CIBPay processes it and sends the money back to the customer's card.

**Settlement:** At the end of the day or week, CIBPay transfers the collected money to the merchant's bank account. This is called settlement.

**Reconciliation:** Comparing CIBPay's records with the bank's records to make sure all transactions match. If there is a difference, it must be investigated.

**Chargeback:** A customer contacts their bank and says "I did not make this payment." The bank takes the money back from the merchant. This is called a chargeback.

**Key Tools / Keywords:** Payment Gateway, Acquiring bank, Issuing bank, Card network (Visa/Mastercard), Settlement, Chargeback, Reconciliation

**Possible Interview Question:**

Q: Can you explain the payment processing workflow?

**Sample Answer:**

Yes. When a customer makes a payment, the process has several steps. First, the merchant sends a payment request to our payment gateway. We validate the request - check the merchant credentials and the payment details. Then we send the payment to the acquiring bank, which forwards it to the card network like Visa or Mastercard. The issuing bank checks if the customer has enough funds and sends back an approval or decline. We receive the result and send it back to the merchant. The whole process usually takes 1-3 seconds. I also understand related workflows like refunds, settlements, and chargebacks. This knowledge helps me identify exactly where a problem occurs when a merchant reports an issue. For example, at Embafinans I implemented a PayTabs card tokenization workflow for credit disbursements to customer cards via Kapital Bank. I managed the full process: card registration with verification, token issuance, and token-based payouts. This hands-on experience gave me deep understanding of how card operations work in practice.

Tip: Use the word "gateway" and mention "tokenization" - it shows you understand card operations hands-on.

TOPIC 8

## Cybersecurity & Data Protection

### What is this?

**Cybersecurity** means protecting systems, data, and networks from attacks. In a payment company, cybersecurity is extremely important because you handle people's money and personal information.

**Data protection** means keeping customer and merchant data safe. This includes: credit card numbers, personal information, transaction history, API keys, and passwords.

Key concepts you should know:

**Encryption:** Converting data into a secret code so only authorized people can read it. For example, HTTPS encrypts all data between the browser and the server.

**Authentication:** Verifying who someone is. For example, API keys, passwords, two-factor authentication (2FA).

**Authorization:** Controlling what someone can do. For example, a merchant can only see their own transactions, not other merchants' transactions.

**PCI-DSS:** Payment Card Industry Data Security Standard. This is a set of security rules that all payment companies must follow. It says: never store full credit card numbers, use encryption, regularly test security systems.

### Why is it important?

A security breach at a payment company is catastrophic. If hackers steal credit card numbers, the company loses reputation, pays huge fines, and can go out of business. As a support specialist, you handle sensitive data every day: API keys, merchant credentials, transaction details. You must follow security best practices to protect this data.

### How does it work?

How to apply security in your daily work:

1. **Never share credentials in plain text.** Always use secure channels.
2. **Use strong passwords and API keys.** Never use default or weak passwords.
3. **Principle of least privilege.** Give each merchant only the access they need, not more.
4. **Always use HTTPS.** Never HTTP (without S). HTTPS encrypts all data.
5. **Monitor for suspicious activity.** If a merchant suddenly makes 1000 transactions instead of their usual 10, investigate immediately.
6. **Follow PCI-DSS rules.** Never store full card numbers, always mask them. For example, show 4XXX XXXX XXXX 1234 instead of the full number.
7. **Report security incidents immediately.** If you see something suspicious, tell your security team right away.

**Key Tools / Keywords:** HTTPS/TLS, Encryption, PCI-DSS, 2FA, Access control, Firewall, API key management

**Possible Interview Question:**

Q: How do you ensure data protection and security in your daily work?

Sample Answer:

Security is a top priority for me. In my daily work, I follow several security practices. First, I never share API keys or credentials through unsecured channels like email. I always use the company's secure credential management system. Second, I follow the principle of least privilege - I give each merchant only the access they need. Third, I always verify that all connections use HTTPS, never plain HTTP. I also monitor for unusual activity - for example, if a merchant's transaction volume suddenly increases dramatically, I investigate to make sure it is legitimate. And of course, I follow PCI-DSS rules: I never store full card numbers, I always mask sensitive data, and I report any security concerns to our security team immediately.

Tip: Always mention "PCI-DSS" in a payment company interview. It shows you know the industry standard.

TOPIC 9

## Technical Documentation & FAQs

### What is this?

**Technical documentation** means writing guides and manuals that help merchants use the API and the payment system. This includes: API integration guides, troubleshooting manuals, error code references, and setup instructions.

**FAQ (Frequently Asked Questions)** is a list of common questions and answers. For example: "Why is my payment failing?" "How do I get my API key?" "What does error code 401 mean?" FAQs help merchants find answers quickly without contacting support.

### Why is it important?

Good documentation reduces the number of support requests. If merchants can find answers themselves, they do not need to contact you. This saves time for both merchants and the support team. Good documentation also makes the onboarding process faster because merchants can read the integration guide and set up by themselves. In a payment company, accurate documentation is critical because payment errors can cause financial loss.

### How does it work?

Types of documentation you should create:

1. **API Integration Guide:** Step-by-step instructions for connecting to the API. Include: endpoint URLs, required headers, request/response examples, error codes.
2. **Test Case Guide:** How to test the integration. Include: test card numbers, test scenarios, expected results.
3. **Error Code Reference:** A table of all error codes with explanations. For example: "401 - Unauthorized: Check your API key."
4. **Troubleshooting Guide:** Common problems and how to fix them. For example: "Payment timeout - Check your network connection and try again."
5. **FAQ:** Questions that merchants ask most often, with clear answers.

Documentation should be: clear, simple, accurate, and up-to-date.

**Key Tools / Keywords:** Confluence, Wiki, Markdown, API documentation (Swagger/OpenAPI), Screenshots, Examples

**Possible Interview Question:**

Q: How do you create technical documentation for merchants?

**Sample Answer:**

I create different types of documentation for merchants. The API integration guide includes endpoint URLs, required headers, JSON examples for requests and responses, and error code explanations. I also create a troubleshooting guide with the most common problems and their solutions. For the FAQ, I collect questions that merchants ask most often and write clear answers. I use simple language and include screenshots and examples so that even non-technical merchants can understand. I also maintain the documentation - when the API changes, I update the docs immediately so merchants always have accurate information.

Tip: Say "simple language and screenshots" - this shows you think about non-technical users.

TOPIC 10

## Client Service & Communication Skills

### What is this?

**Client service** means helping merchants with their problems in a friendly, patient, and professional way. In a payment company, the IT Support Specialist is the primary contact for merchants. Merchants are not always technical people. They might be business owners, shop managers, or accountants. They do not understand API, JSON, or HTTP codes. Your job is to explain technical problems in simple language they can understand.

**Communication skills** means you can: listen carefully to the merchant's problem, ask the right questions to understand the issue, explain the solution clearly, and follow up to make sure the problem is resolved.

This role is 50% technical and 50% communication. You can be the best technician in the world, but if you cannot explain things to merchants clearly, you will not be successful in this role.

### Why is it important?

The vacancy says: "bridge the gap between complex payment technology and exceptional client service." This means CIBPay wants someone who is BOTH technical AND good with people. Many technical people are not good at communication. If you can show both skills, you have a big advantage over other candidates. Good communication also reduces the number of support requests because merchants understand your explanations and can fix simple problems themselves next time.

### How does it work?

How to communicate effectively with merchants:

1. **Listen first:** Let the merchant explain their problem completely before you speak. Do not interrupt.
2. **Ask clarifying questions:** "Can you share the error message?" "What time did this happen?" "Which endpoint are you calling?"

3. **Explain in simple language:** Instead of saying "The API returned a 401 Unauthorized due to an invalid bearer token," say: "Your API key is not correct. Please check your API key in the back-office and try again."
4. **Provide step-by-step instructions:** Break the solution into small steps. Use numbered lists. Include screenshots when possible.
5. **Follow up:** After the fix, contact the merchant to confirm everything is working. This shows you care about their success.
6. **Stay calm under pressure:** Merchants can be angry when payments are not working. Remember: they are not angry at you personally. Stay professional and solution-focused.
7. **Use the right channel:** Email for detailed explanations with screenshots. Chat for quick questions. Phone calls for urgent or complex issues.

**Key Tools / Keywords:** Email, Chat, Phone, Ticket system, Screenshots, Step-by-step guides, Empathy, Patience

**Possible Interview Question:**

Q: How do you handle a frustrated merchant who is angry about a payment issue?

**Sample Answer:**

First, I listen carefully and let them explain the problem without interrupting. I understand that payment issues are stressful for merchants because it affects their business and their customers. I do not take their frustration personally. Then I apologize for the inconvenience and assure them I will help. I ask for specific details: the error message, the time, and the transaction ID. I explain what I am going to do step by step, so they know I am taking action. I keep them updated on progress. When the issue is resolved, I follow up to confirm everything is working and ask if they need anything else. This approach usually turns a frustrated merchant into a satisfied one. In my experience, patience and clear communication are key. For example, at Embafinans, I regularly communicated with external partners like Kapital Bank and PayTabs support teams to resolve transaction failures. I always kept all parties informed and followed up until the issue was completely resolved.

Tip: Show empathy: say "I understand how important this is for your business." Mention experience with external partners - it shows you can communicate beyond your own team.

TOPIC 11

## Payment Error Codes & Card Operations

### What is this?

When a payment fails, the system returns an error code. As a support specialist, you must know what each error code means and how to help the merchant fix it.

**Common API Error Codes:**
**200 - Success:** The payment was processed successfully.
**400 - Bad Request:** The merchant sent incorrect data. For example: missing required field, wrong format, invalid amount.
**401 - Unauthorized:** The API key is wrong, expired, or missing.
**403 - Forbidden:** The merchant does not have permission for this action.

**404 - Not Found:** The URL or resource does not exist.
**408 - Request Timeout:** The request took too long. The merchant's server or network might be slow.
**429 - Too Many Requests:** The merchant is sending too many requests too fast. They need to slow down.
**500 - Internal Server Error:** Something is wrong on CIBPay's side. Escalate to the development team.
**502 / 503 - Service Unavailable:** The server is temporarily down or under maintenance.

**Common Card Decline Reasons:**
**Insufficient funds:** The customer does not have enough money in their account.
**Expired card:** The customer's card has expired.
**Incorrect CVV:** The 3-digit security code on the back of the card is wrong.
**Card blocked:** The customer's bank has blocked the card for security reasons.
**Transaction limit exceeded:** The payment amount is higher than the card's limit.

**Important concepts:**
**3-D Secure (3DS):** An extra security step where the customer must enter a one-time password (OTP) sent by their bank. This is required for online payments in many countries.
**Card Tokenization:** Replacing the real card number with a random token (like 4XXX-XXXX-XXXX-1234). This makes transactions safer because the real card number is never stored.

### Why is it important?

Merchants will call you every day asking: "Why did this payment fail?" If you can quickly look at the error code and explain the reason, you solve the problem fast. If you do not know the error codes, you have to ask someone else, which wastes time and makes the merchant lose trust in you. Knowing card decline reasons is also important because merchants often ask: "The customer says they have money, why was the card declined?" You need to explain the possible reasons clearly.

### How does it work?

How to handle error reports from merchants:

1. **Ask for the error code:** "Can you share the exact error message or code you received?"
2. **Look up the error:** Check your error code reference table.
3. **Determine the cause:** Is it a merchant-side issue (400, 401, 408) or a server-side issue (500, 502, 503)?
4. **Explain to the merchant:** Tell them what the error means in simple language and what they need to do to fix it.
5. **For card declines:** The merchant cannot fix these. Explain that the customer needs to contact their bank or use a different card.
6. **For server errors:** Escalate to the development team immediately. Tell the merchant: "Our team is working on this. I will update you shortly."

**Key Tools / Keywords:** HTTP status codes, Card decline reasons, 3-D Secure (3DS), OTP, Card tokenization, Error code reference table

**Possible Interview Question:**

Q: A merchant says a payment was declined. How do you investigate?

**Sample Answer:**

First, I ask the merchant for the transaction ID and the error code. If the error code is a client-side error like 400, I check the request data - maybe the amount is invalid or a required field is missing. If it is a 401, I check their API key. If the payment was declined by the bank, I check the decline reason code. Common reasons are: insufficient funds, expired card, incorrect CVV, or card blocked. I explain the reason to the merchant and advise them to ask the customer to contact their bank or try a different payment method. If I see a 500 or 503 error, I know it is a server-side issue and I immediately escalate to our development team while keeping the merchant updated on the progress. In my experience at Embafinans, when card transactions failed during PayTabs payouts, I contacted both the Kapital Bank support team and the PayTabs support team to find the root cause. This cross-team coordination helped us resolve issues faster.

Tip: Mention that you have real experience coordinating with bank support teams - this is very valuable for a payment company.

---

## FINAL INTERVIEW TIPS

1. **Use the STAR Method:** S (Situation) = What was the problem? T (Task) = What did you need to do? A (Action) = What did YOU do? R (Result) = What was the outcome? Use this structure for every answer.

2. **Use Numbers:** "300-500 daily transactions" is better than "many transactions." Numbers show measurable impact.

3. **Connect Everything to Your Experience:** When they ask about theory, always give an example from your work at Embafinans, Birbonus, or Umico.

4. **Be Honest:** If you do not know something, say: "I have basic knowledge of this and I am learning more." Then connect it to what you DO know.

5. **Speak Confidently:** Your 15+ years of engineering background is a big advantage. Not many IT Support specialists have deep technical knowledge.

6. **Ask Questions:** At the end, ask about the team, current challenges, and the tools they use. This shows interest and preparation.

Remember: You do not need to be perfect. You need to show that you can learn quickly and solve problems. Good luck!